

Documentation Technique

Architecture de l'application

L'application est composee de trois parties principales :

- Frontend : Angular 17
- Backend : Flask (Python)
- Base de donnees : MariaDB

L'application utilise Docker pour faciliter le deploiement avec 3 conteneurs :

- Conteneur web (Angular)
- Conteneur api (Flask)
- Conteneur database (MariaDB)

Technologies utilisees

Frontend

- Angular 17 (framework JavaScript)
- Angular Material (composants UI)
- TypeScript
- SCSS pour le style
- i18n pour la traduction (francais/anglais)

Backend

- Python 3
- Flask (framework web)
- Flask-CORS (gestion des requetes cross-origin)

Base de donnees

- MariaDB

Outils

- Docker et Docker Compose
- npm (gestionnaire de packages)

Structure des dossiers

Frontend (dossier parc/)

```
parc/  
├─ src/  
│  └─ app/
```

```

├── accueil/           # Page d'accueil avec liste des attractions
├── admin/             # Page d'administration
├── login/             # Page de connexion
├── critique-dialog/   # Dialog pour ajouter une critique
├── Service/           # Services pour les appels API
├── Interface/         # Interfaces TypeScript
├── Interceptor/       # Intercepteur pour l'authentification
├── locale/            # Fichiers de traduction
├── styles.scss        # Styles globaux
├── angular.json       # Configuration Angular
└── package.json       # Dependances npm

```

Backend (dossier python/)

```

python/
├── controller/
│   ├── attraction.py   # Logique metier pour les attractions
│   ├── critique.py    # Logique metier pour les critiques
│   └── auth/
│       └── auth.py     # Gestion de l'authentification
├── request/
│   └── request.py      # Fonctions de connexion a la base de donnees
├── sql_file/
│   ├── init.sql       # Creation des tables
│   └── create.sql     # Donnees initiales
└── app.py              # Point d'entree de l'API

```

Fonctionnement de l'application

Demarrage

1. Docker Compose lance les 3 conteneurs
2. La base de donnees est initialisee avec les tables
3. Le backend Flask démarre sur le port 5000
4. Le frontend Angular démarre sur le port 4200

Flux de donnees

Affichage des attractions (visiteur)

1. Le composant Accueil appelle `attractionService.getVisibleAttractions()`
2. Le service fait une requete GET vers `/attraction/visible`
3. Le backend interroge la base de donnees avec `SELECT * FROM attraction WHERE visible = 1`
4. Les attractions visibles sont retournees en JSON
5. Le frontend affiche les attractions dans des cartes Material

Ajout d'une critique

1. L'utilisateur clique sur "Laisser un avis"
2. Un dialog s'ouvre avec le formulaire
3. L'utilisateur remplit le formulaire (texte, note, nom/prenom optionnel)
4. A la validation, le service envoie une requete POST vers `/critique`
5. Le backend valide les donnees et les insere dans la base
6. Le frontend recharge les critiques de l'attraction

Administration des attractions

1. L'administrateur se connecte via la page Login
2. Le backend genere un token JWT
3. Le token est stocke dans le localStorage
4. L'intercepteur ajoute le token dans toutes les requetes protegees
5. L'admin peut ajouter/modifier des attractions
6. Les requetes POST/DELETE verifient le token avant execution

API REST

Endpoints publics

- `GET /attraction/visible` - Liste des attractions visibles
- `POST /critique` - Ajouter une critique
- `GET /critique/<attraction_id>` - Critiques d'une attraction
- `POST /login` - Connexion administrateur

Endpoints proteges (token requis)

- `GET /attraction` - Toutes les attractions
- `POST /attraction` - Ajouter une attraction
- `DELETE /attraction/<id>` - Supprimer une attraction

Systeme de traduction

L'application utilise Angular i18n :

- Les textes dans les templates ont l'attribut `i18n`
- La commande `ng extract-i18n` extrait les textes
- Les traductions sont dans `src/locale/messages.en.xlf`
- Le build genere deux versions : `dist/parc/browser/fr/` et `dist/parc/browser/en/`

Gestion de l'authentification

Le backend utilise JWT (JSON Web Token) :

1. Le login renvoie un token encode
2. Le token contient l'ID utilisateur et expire apres un certain temps
3. L'intercepteur Angular ajoute le header `Authorization: Token <token>`
4. Le backend decode et verifie le token pour chaque requete protegee

Base de donnees

Connexion

- Host : database (nom du conteneur Docker)
- Port : 3306
- User : mysqlusr
- Password : mysqlpwd
- Database : parc

Tables

Voir le document "schema_base_de_donnees.md"

Securite

Points implementes

- Authentification par token JWT
- CORS configure pour accepter les requetes du frontend
- Validation des donnees cote backend
- Foreign keys avec CASCADE pour l'integrite referentielle

Points a ameliorer

- Hachage des mots de passe (actuellement en clair)
- Protection contre les injections SQL (utilisation de requetes parametrees)
- HTTPS en production
- Gestion de l'expiration des tokens

Commandes utiles

Demarrer l'application

```
docker compose up -d
```

Arreter l'application

```
docker compose down
```

Rebuild les conteneurs

```
docker compose down -v  
docker compose up -d --build
```

Voir les logs

```
docker logs parcattraction-master-web-1    # Frontend
docker logs parcattraction-master-api-1    # Backend
docker logs parcattraction-master-database-1 # Database
```

Build de production

```
cd parc
npm run build
# ou avec traduction
ng build --localize
```